



INDIAN INSTITUTE OF TECHNOLOGY DELHI

COL 729 Lab 2: IR and Compiler Passes

Submitted by

Aditya Anand and Varun Subramaniam
2023CS50284 and 2023CS50497

Instructor

Prof. Sorav Bansal

9 September 2026

Machines used

Parts A and B were done on one machine. Parts C and D were done on another. The two Homebrew LLVM installs are not the same version.

Parts A and B. Apple M3, arm64 macOS. `opt` and `clang` were Homebrew LLVM 23.1.0.

```
Homebrew LLVM version 23.1.0
Optimized build.
Default target: arm64-apple-darwin25.5.0
Host CPU: apple-m3
```

Parts C and D. Apple M1, arm64 macOS. `opt` and `clang` were Homebrew LLVM 18.1.8.

```
Homebrew LLVM version 18.1.8
Optimized build.
Default target: arm64-apple-darwin25.5.0
Host CPU: apple-m1
```

1 Part A

All except `select` are taken from `opcodes.00.ll`. `select` is from `opcodes.canonical.ll`.

`alloca`

```
13: %1 = alloca i32, align 4
```

This line creates a space for a 32-bit integer on the stack (for `completimeindex`) and `%1` is its address. The result type is `ptr`.

`load`

```
16: %3 = load volatile i32, ptr @runtimeindex, align 4
```

This line loads a 32-bit integer *from* `@runtimeindex` and writes that value into the SSA name `%3`. It does not store into `runtimeindex`. The result type is `i32`.

`store`

```
15: store i32 0, ptr %1, align 4
```

This stores the value 0 into the memory that `%1` points to (`completimeindex`). `%1` is only the address and it does not change. `store` has no result.

`getelementptr`

```
18: %5 = getelementptr inbounds [2 x %struct.myStruct], ptr %2, i64 0, i64 %4
```

`%2` gives the base address of the `myStruct` array. The first `i64 0` moves the pointer by 0 distance forward and changes the pointing type from a hypothetical array of `[2 x %struct.myStruct]` and points to a `[2 x %struct.myStruct]` without moving. The second `i64 %4` does pointer arithmetic with the offset stored in `%4` and adds that as offset, so `%5` is the required address. The result type is `ptr`. This is done in the line `arr[runtimeindex].scalar = 1`. The `getelementptr` arithmetic is purely done on the addresses. There is no memory access involved. The memory access is done by a separate load/store. Hence, `getelementptr` does not read the memory.

icmp

```
46: %5 = icmp slt i32 %4, 3
```

This is for integer comparison. It sets %5 to true if %4 is less than 3. That is the loop continuation test `i < 3` in `f2`: true keeps going around the loop, false leaves it. It is not the `break`. The result type is `i1`.

fcmp

```
67: %18 = fcmp oeq double %17, 2.000000e+00
```

`oeq` means neither operand is NaN and the two values are equal. The C is `else if (myf == 2.0) break;` in `f2`. The literal is 2.0, a `double`, so `myf` is first widened with `fext` and then compared as `fcmp oeq double`. That is why this is not a float compare against `2.0f`. The result type is `i1`.

add

```
51: %8 = add nsw i32 %7, 1
```

`nsw` means no signed wrap, meaning that no overflow will take place. In this line 1 is added to %7 to get %8.

This is the `loopcarried += 1` step inside the loop in `f2`. %7 is the value loaded from `loopcarried` on line 50. The result type is `i32`.

fdiv

```
55: %11 = fdiv float %10, 2.000000e+00
```

The float division value of %10 divided by `2.0f` is stored in %11 as done in `f2`. The result type is `float`.

br

```
42: br label %3
```

This is the jump from the start of `f2` into the loop header of `for (int i = 0; i < 3; i++)`. The header is block %3. `br` has no result.

switch

```
101: switch i32 %6, label %31 [
```

This is `switch(a)` in `f3`. %6 is the loaded value of `a`. If no case matches, control goes to block %31, the join after the switch. `switch` has no result.

phi

```
120: %16 = phi i32 [ %13, %12 ], [ 7, %14 ]
```

It changes this line `int t = (runtimeindex > 1) ? loopcarried : 7;`. Clang emits it as 4 basic blocks %9, %12, %14, %15; the phi does not create them. The phi sits at the join block %15 and picks %13 if control came from %12 and 7 if it came from %14, so that SSA is not broken. The result type is `i32`.

call

```
104: call void @f1()
```

Calls the function `f1`. It is the first case of the switch. `f1` returns void so there is no result.

ret

```
34: ret void
```

This is the implicit return at the end of void `f1`. There is no `return` keyword in the C; clang still emits `ret void`. `ret` has no result.

sext

```
17: %4 = sext i32 %3 to i64
```

It is a `sext` (signed extension) of the value of %3 from 32 to 64 bits because pointer arithmetic is done in 64 bits. The result type is `i64`.

sitofp

```
54: %10 = sitofp i32 %9 to float
```

It is used in this line `float myf = loopcarried/2.0f;`. %9 is the `loopcarried` value, which is converted into a float register %10 before doing arithmetic. The result type is `float`.

select

```
71: %4 = select i1 %2, i32 %3, i32 7
```

It changes this line `int t = (runtimeindex > 1) ? loopcarried : 7;`. Instead of using 2 basic blocks to first calculate the values and using phi, it is storing the result of the comparison in %2 and using its result directly to take from one of the branches. The result type is `i32`.

Control flow graph of f2

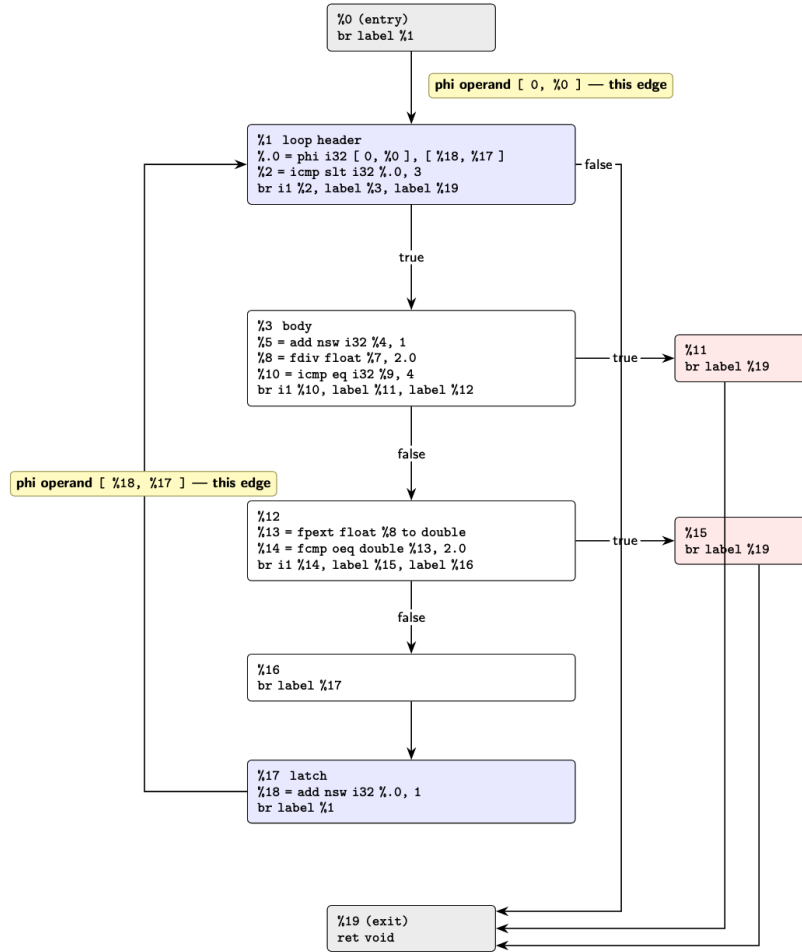


Figure 1: f2 after mem2reg (opt -S -passes=mem2reg opcodes.00.11).

After mem2reg: struct access

These two GEPs are from opcodes.mem2reg.11 (opt -S -passes=mem2reg opcodes.00.11). After mem2reg, compiletimeindex's alloca is gone, so arr is %1.

```

%4 = getelementptr @inbounds [2 x %struct.myStruct], ptr %1, i64 0, i64 %3
%5 = getelementptr @inbounds %struct.myStruct, ptr %4, i32 0, i32 0

```

The layout of myStruct on this target is: scalar at offset 0, array[2][2] at bytes 4–19, 4 bytes of padding so the pointer is aligned, ptr at offset 24, total size 32, alignment 8.

First GEP: %1 is the base of the arr allocation. i64 0 treats that pointer as pointing at one [2 x %struct.myStruct] and does not move it. i64 %3 is the run-time index runtimeindex after sext. That steps by the struct size, so the address is base + 32 * runtimeindex. Result: pointer to arr[runtimeindex].

Second GEP: the first i32 0 is “this struct, no extra stride”. The second i32 0 is field 0, which is .scalar. That field is at offset 0, not 32× anything. 32 bytes is only the size of one myStruct, used by the array-element index above.

The `getelementptr` arithmetic is purely done on the addresses. There is no memory access involved. The memory access is done by a separate load/store. Hence, `getelementptr` does not read the memory.

The 2D field

`arr[runtimeindex].array[compiletimeindex][compiletimeindex]` with both compile-time indices equal to 0. After `mem2reg`:

```
%8 = getelementptr inbounds [2 x %struct.myStruct], ptr %1, i64 0, i64 %7
%9 = getelementptr inbounds %struct.myStruct, ptr %8, i32 0, i32 1
%10 = getelementptr inbounds [2 x [2 x i32]], ptr %9, i64 0, i64 0
%11 = getelementptr inbounds [2 x i32], ptr %10, i64 0, i64 0
store i32 2, ptr %11, align 4
```

Same layout as above: one `myStruct` is 32 bytes, `array` starts at byte 4, each row is 8 bytes, each `int` is 4 bytes.

GEP	What it selects	Extra bytes
i64 %7	<code>arr[runtimeindex]</code>	$32 \times \text{runtimeindex}$
i32 0, i32 1	field 1, <code>.array</code>	+4
i64 0, i64 0	row 0 of <code>[2][2]</code>	+0 (row 1 would be +8)
i64 0, i64 0	column 0 of that row	+0 (column 1 would be +4)

So the address is `base + 32 · runtimeindex + 4`. For `runtimeindex = 1` that is 36 bytes past the start of `arr`. The `store i32 2` is the actual write; the GEPs only built the pointer.

2 Part B

Counts and named pipelines here are from Homebrew LLVM 23.1.0.

function	opt	blocks	br	ret	switch	add	alloca	call	fcmp	fdiv	fmul	fpext	getelementptr	icmp	load	phi	select	sext	sitofp	store	trunc
f1	-O0	1	0	1	0	0	2	0	0	0	0	0	9	0	3	0	0	3	0	4	0
f1	-O1	1	0	1	0	0	1	2	0	0	0	0	3	0	3	0	0	3	0	3	0
f1	-O2	1	0	1	0	0	0	0	0	0	0	0	0	0	3	0	0	0	0	0	0
f1	-O3	1	0	1	0	0	0	0	0	0	0	0	0	0	3	0	0	0	0	0	0
f2	-O0	9	8	1	0	2	2	0	1	1	0	1	0	2	6	0	0	0	1	4	0
f2	-O1	5	4	1	0	3	0	0	1	0	1	0	0	2	1	3	0	0	1	1	0
f2	-O2	6	5	1	0	3	0	0	2	0	2	0	0	2	1	1	0	0	2	1	0
f2	-O3	6	5	1	0	3	0	0	2	0	2	0	0	2	1	1	0	0	2	1	0
f3	-O0	9	7	1	1	1	6	2	0	0	0	0	0	1	9	1	0	1	1	8	1
f3	-O1	1	0	1	0	1	0	1	0	0	0	0	0	1	2	0	1	0	0	1	0
f3	-O2	1	0	1	0	1	0	1	0	0	0	0	0	1	2	0	1	0	0	1	0
f3	-O3	1	0	1	0	1	0	1	0	0	0	0	0	1	2	0	1	0	0	1	0
main	-O0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
main	-O1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
main	-O2	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
main	-O3	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 2: Opcode counts per function and optimisation level.

1)

Before:

```
%1 = alloca i32, align 4
store i32 0, ptr %1, align 4
br label %3

3:
%4 = load i32, ptr %1, align 4
%5 = icmp slt i32 %4, 3
br i1 %5, label %6, label %24
```

```

21:
%22 = load i32, ptr %1, align 4
%23 = add nsw i32 %22, 1
store i32 %23, ptr %1, align 4
br label %3, !llvm.loop !5

```

After opt -S -passes=mem2reg opcodes.00.ll:

```

br label %1

1:
%.0 = phi i32 [ 0, %0 ], [ %18, %17 ]
%2 = icmp slt i32 %.0, 3
br i1 %2, label %3, label %19

17:
%18 = add nsw i32 %.0, 1
br label %1, !llvm.loop !5

```

Explanation: Change made by mem2reg. In f2, the loop counter becomes a phi node. Earlier the loop counter was in the stack slot. Now it is promoted to a register. After the compare, the branch goes into the loop body if the counter is still less than 3, and leaves the loop otherwise. The increment adds one to that phi value and jumps back to the header, with no store back to the stack.

2)

f3 switches on int a = 1. At -00 the switch and all four cases are still there (9 blocks, one switch). Cases 0, 2 and 3 are dead.

Before (opcodes.00.ll):

```

store i32 1, ptr %1, align 4
%6 = load i32, ptr %1, align 4
switch i32 %6, label %31 [
    i32 0, label %7
    i32 1, label %8
    i32 2, label %19
    i32 3, label %25
]

```

After opt -S -passes='function(mem2reg,sccp,simplifcfg)':

```

%1 = load volatile i32, ptr @runtimeindex, align 4
%2 = icmp sgt i32 %1, 1
%3 = load i32, ptr @loopcarried, align 4
%4 = select i1 %2, i32 %3, i32 7
%5 = load i32, ptr @loopcarried, align 4
%6 = add nsw i32 %5, %4
store i32 %6, ptr @loopcarried, align 4
call void @f1()
ret void

```

sccp sees that the switch condition is the constant 1. simplifcfg deletes the unused arms and the switch. What is left is only case 1: the ternary becomes a select, then call @f1. That is why the Part B table goes from 9 blocks to 1 and from one switch to zero at -01.

3)

Before:

```
3:                                     ; preds = %1
%9 = load i32, ptr @loopcarried, align 4
%10 = icmp eq i32 %9, 4
br i1 %10, label %11, label %12

11:                                   ; preds = %3
br label %19

12:                                   ; preds = %3
%13 = fpext float %8 to double
%14 = fcmp oeq double %13, 2.000000e+00
br i1 %14, label %15, label %16

15:                                   ; preds = %12
br label %19

16:                                   ; preds = %12
br label %17
```

After:

```
3:                                     ; preds = %1
%9 = load i32, ptr @loopcarried, align 4
%10 = icmp eq i32 %9, 4
%11 = fpext float %8 to double
%12 = fcmp oeq double %11, 2.000000e+00
%or.cond = select i1 %10, i1 true, i1 %12
br i1 %or.cond, label %15, label %13
```

The before code is after `opt -S -passes=mem2reg opcodes.00.11`. The after code is after `opt -S -passes='mem2reg,simplifcfg' opcodes.00.11`.

It collapses the 2 statements in the code:

```
if (loopcarried == 4) break;
else if (myf == 2.0) break;
```

into a common `or.cond` check, simplifying the branch specification.

3 Part C

These numbers are from Homebrew LLVM 18.1.8, the same `opt` and `clang`.

The default pipelines were printed with:

```
opt -passes="default<01>" -print-pipeline-passes -disable-output opcodes.00.11
```

The same command was run for 00, 02 and 03. `opt -print-passes` was used to see which names are real transforms and which are only analyses.

The dump is not only transforms. It also wraps passes in `function` and `loop`, and it prints analyses such as `require`, `invalidate` and `verify`. Those were left out of the count. There is also an `AArch64` class name that `opt` will not take as a pass, so that was left out too. If the same transform ran more than once each run was kept in the order it appeared.

Counts

Level	Invocations	Distinct
-00	5	5
-01	81	55
-02	97	66
-03	100	69

Ordered lists

Passes in the order they ran.

-00	-01	-02	-03
1. always-inline	1. annotation2metadata	1. annotation2metadata	1. annotation2metadata
2. coro-early	2. forceattrs	2. forceattrs	2. forceattrs
3. coro-split	3. inferattrs	3. inferattrs	3. inferattrs
4. coro-cleanup	4. coro-early	4. coro-early	4. coro-early
5. globaldce	5. lower-expect	5. lower-expect	5. lower-expect
	6. simplifycfg	6. simplifycfg	6. simplifycfg
	7. sroa	7. sroa	7. sroa
	8. early-cse	8. early-cse	8. early-cse
	9. openmp-opt	9. openmp-opt	9. callsite-splitting
	10. ipscpp	10. ipscpp	10. openmp-opt
	11. called-value-propagation	11. called-value-propagation	11. ipscpp
	12. globalopt	12. globalopt	12. called-value-propagation
	13. mem2reg	13. mem2reg	13. globalopt
	14. instcombine	14. instcombine	14. mem2reg
	15. simplifycfg	15. simplifycfg	15. instcombine
	16. always-inline	16. always-inline	16. simplifycfg
	17. inline	17. inline	17. always-inline
	18. function-attrs	18. function-attrs	18. inline
	19. sroa	19. openmp-opt-cgsc	19. function-attrs
	20. early-cse	20. sroa	20. argpromotion
	21. simplifycfg	21. early-cse	21. openmp-opt-cgsc
	22. instcombine	22. speculative-execution	22. sroa
	23. libcalls-shrinkwrap	23. jump-threading	23. early-cse
	24. simplifycfg	24. correlated-propagation	24. speculative-execution
	25. reassociate	25. simplifycfg	25. jump-threading
	26. loop-instsimplify	26. instcombine	26. correlated-propagation
	27. loop-simplifycfg	27. aggressive-instcombine	27. simplifycfg
	28. licm	28. libcalls-shrinkwrap	28. instcombine
	29. loop-rotate	29. tailcallelim	29. aggressive-instcombine
	30. licm	30. simplifycfg	30. libcalls-shrinkwrap
	31. simple-loop-unswitch	31. reassociate	31. tailcallelim
	32. simplifycfg	32. constraint-elimination	32. simplifycfg
	33. instcombine	33. loop-instsimplify	33. reassociate
	34. loop-idiom	34. loop-simplifycfg	34. constraint-elimination
	35. indvars	35. licm	35. loop-instsimplify
	36. loop-deletion	36. loop-rotate	36. loop-simplifycfg
	37. loop-unroll-full	37. licm	37. licm
	38. sroa	38. simple-loop-unswitch	38. loop-rotate
	39. memcpypopt	39. simplifycfg	39. licm
	40. sccp	40. instcombine	40. simple-loop-unswitch
	41. bdce	41. loop-idiom	41. simplifycfg
	42. instcombine	42. indvars	42. instcombine
	43. coro-elide	43. loop-deletion	43. loop-idiom
	44. adce	44. loop-unroll-full	44. indvars
	45. simplifycfg	45. sroa	45. loop-deletion
	46. instcombine	46. vector-combine	46. loop-unroll-full
	47. function-attrs	47. mldst-motion	47. sroa
	48. coro-split	48. gvn	48. vector-combine
	49. deadargelim	49. sccp	49. mldst-motion
	50. coro-cleanup	50. bdce	50. gvn
	51. globalopt	51. instcombine	51. sccp
	52. globaldce	52. jump-threading	52. bdce
	53. elim-avail-extern	53. correlated-propagation	53. instcombine
	54. rpo-function-attrs	54. adce	54. jump-threading
	55. float2int	55. memcpypopt	55. correlated-propagation
	56. lower-constant-intrinsics	56. dse	56. adce
	57. loop-rotate	57. move-auto-init	57. memcpypopt
	58. loop-deletion	58. licm	58. dse
	59. loop-distribute	59. coro-elide	59. move-auto-init
	60. inject-tli-mappings	60. simplifycfg	60. licm
	61. loop-vectorize	61. instcombine	61. coro-elide
	62. infer-alignment	62. function-attrs	62. simplifycfg
	63. loop-load-elim	63. coro-split	63. instcombine
	64. instcombine	64. deadargelim	64. function-attrs
	65. simplifycfg	65. coro-cleanup	65. coro-split
	66. vector-combine	66. globalopt	66. deadargelim
	67. instcombine	67. globaldce	67. coro-cleanup
	68. loop-unroll	68. elim-avail-extern	68. globalopt
	69. sroa	69. rpo-function-attrs	69. globaldce
	70. infer-alignment	70. float2int	70. elim-avail-extern
	71. instcombine	71. lower-constant-intrinsics	71. rpo-function-attrs
	72. licm	72. loop-rotate	72. float2int

-00	-01	-02	-03
	73. alignment-from-assumptions	73. loop-deletion	73. lower-constant-intrinsics
	74. loop-sink	74. loop-distribute	74. chr
	75. instsimplify	75. inject-tli-mappings	75. loop-rotate
	76. div-rem-pairs	76. loop-vectorize	76. loop-deletion
	77. tailcallelim	77. infer-alignment	77. loop-distribute
	78. simplifcfg	78. loop-load-elim	78. inject-tli-mappings
	79. globaldce	79. instcombine	79. loop-vectorize
	80. constmerge	80. simplifcfg	80. infer-alignment
	81. rel-lookup-table-converter	81. slp-vectorizer	81. loop-load-elim
		82. vector-combine	82. instcombine
		83. instcombine	83. simplifcfg
		84. loop-unroll	84. slp-vectorizer
		85. sroa	85. vector-combine
		86. infer-alignment	86. instcombine
		87. instcombine	87. loop-unroll
		88. licm	88. sroa
		89. alignment-from-assumptions	89. infer-alignment
		90. loop-sink	90. instcombine
		91. instsimplify	91. licm
		92. div-rem-pairs	92. alignment-from-assumptions
		93. tailcallelim	93. loop-sink
		94. simplifcfg	94. instsimplify
		95. globaldce	95. div-rem-pairs
		96. constmerge	96. tailcallelim
		97. rel-lookup-table-converter	97. simplifcfg
			98. globaldce
			99. constmerge
			100. rel-lookup-table-converter

Pass descriptions

Pass	Description	-00	-01	-02	-03
adce	Removes instructions whose results are never used	0	1	1	1
aggressive-instcombine	Extra peephole simplifications, only worth running from -02	0	0	1	1
alignment-from-assumptions	Improves load/store alignment using <code>llvm.assume</code> hints	0	1	1	1
always-inline	Inlines functions marked <code>always_inline</code>	1	1	1	1
annotation2metadata	Turns source annotations into IR metadata	0	1	1	1
argpromotion	Replaces a read-only pointer argument with its value	0	0	0	1
bdce	Removes individually unused bits of a value	0	1	1	1
called-value-propagation	Figures out the target of an indirect call when possible	0	1	1	1
callsite-splitting	Splits a call so each caller path can be optimised separately	0	0	0	1
chr	Straightens out a chain of hot/cold branches	0	0	0	1
constmerge	Merges multiple global constants that have identical values into one	0	1	1	1
constraint-elimination	Removes comparisons already implied by an earlier one	0	0	1	1
coro-cleanup	Final cleanup step for coroutines	1	1	1	1
coro-early	Early setup step for coroutines	1	1	1	1
coro-elide	Skips allocating a coroutine frame when not needed	0	1	1	1
coro-split	Splits a coroutine into its resume/destroy/cleanup parts	1	1	1	1
correlated-propagation	Reuses facts proven true earlier in a branch	0	0	2	2
deadargelim	Removes unused function arguments	0	1	1	1
div-rem-pairs	Keeps a matching div and rem together for one instruction	0	1	1	1

Pass	Description	-00	-01	-02	-03
dse	Removes a store that gets overwritten before it's read	0	0	1	1
early-cse	Removes duplicate computations early, before loop passes run	0	2	2	2
elim-avail-extern	Drops function bodies that are no longer needed locally	0	1	1	1
float2int	Does float work as integers when the result is the same	0	1	1	1
forceattrs	Applies attributes passed on the command line	0	1	1	1
function-attrs	Marks functions as <code>readonly</code> , <code>nocapture</code> , and similar	0	2	2	2
globaldce	Removes globals and functions that are never used	1	2	2	2
globalopt	Simplifies and shrinks global variables	0	2	2	2
gvn	Removes a computation that duplicates an earlier one	0	0	1	1
indvars	Simplifies loop counters into a standard form	0	1	1	1
infer-alignment	Works out stronger alignment for loads and stores	0	2	2	2
inferattrs	Adds known attributes to standard library calls	0	1	1	1
inject-tli-mappings	Notes which library calls have a vector version	0	1	1	1
inline	The main inliner, based on a cost heuristic	0	1	1	1
instcombine	General peephole simplifier; runs many times	0	8	8	8
instsimplify	Simplifies instructions without creating new ones	0	1	1	1
ipsccp	Propagates constants across function calls	0	1	1	1
jump-threading	Skips a branch whose outcome is already known	0	0	2	2
libcalls-shrinkwrap	Guards an expensive library call with a cheap check	0	1	1	1
licm	Moves loop-invariant work out of the loop	0	3	4	4
loop-deletion	Removes loops that have no effect	0	2	2	2
loop-distribute	Splits one loop into two independent loops	0	1	1	1
loop-idiom	Replaces a copy-style loop with <code>memcpy</code>	0	1	1	1
loop-instsimplify	Runs instruction simplification inside loops	0	1	1	1
loop-load-elim	Avoids reloading a value already stored last iteration	0	1	1	1
loop-rotate	Turns a while-style loop into a do-while-style loop	0	2	2	2
loop-simplifycfg	Simplifies the control flow inside a loop	0	1	1	1
loop-sink	Moves work into the part of the loop that needs it	0	1	1	1
loop-unroll	Unrolls a loop a few times	0	1	1	1
loop-unroll-full	Fully unrolls a loop with a small constant trip count	0	1	1	1
loop-vectorize	Rewrites a loop to use vector instructions	0	1	1	1
lower-constant-intrinsics	Expands intrinsics like <code>llvm.objectsize</code> into constants	0	1	1	1
lower-expect	Turns <code>llvm.expect</code> hints into branch weights	0	1	1	1
mem2reg	Promotes stack slots into SSA registers	0	1	1	1

Pass	Description	-00	-01	-02	-03
memcpyopt	Replaces copy-style load/store sequences with <code>memcpy</code>	0	1	1	1
mldst-motion	Hoists a load/store shared by both if/else branches	0	0	1	1
move-auto-init	Moves variable-init memsets off the hot path	0	0	1	1
openmp-opt	Optimises OpenMP calls (not used here)	0	1	1	1
openmp-opt-cgsc	OpenMP optimisation run alongside the inliner	0	0	1	1
reassociate	Reorders <code>+</code> / <code>*</code> chains to expose more folding	0	1	1	1
rel-lookup-table-converter	Makes switch jump tables use relative offsets	0	1	1	1
rpo-function-attrs	Infers function attributes across the call graph	0	1	1	1
sccp	Folds a value to a constant along reachable branches only	0	1	1	1
simple-loop-unswitch	Pulls a loop-invariant branch out of the loop	0	1	1	1
simplifycfg	Fewer blocks and branches; runs many times	0	8	8	8
slp-vectorizer	Vectorises independent scalar ops within one block	0	0	1	1
speculative-execution	Hoists a cheap op above a branch when safe	0	0	1	1
sroa	Breaks a struct/array alloca into separate scalars	0	4	4	4
tailcallelim	Turns a tail call into a jump back to the start	0	1	2	2
vector-combine	Cleans up vector instructions after vectorisation	0	1	2	2

1)

i) transformations common to all four levels:

Only these five names appear at -00, -01, -02 and -03. -00 is just always-inline, the coroutine lowering, and a global DCE. Nothing like mem2reg or instcombine runs there.

```
always-inline, coro-early, coro-split, coro-cleanup, globaldce
```

2)

ii) transformations first introduced at O1, O2, or O3:

First introduced at -01. This is where the real pipeline starts. The rest of the O1 ordered list above is also new here; it is just not repeated in the box.

```
mem2reg, sroa, instcombine, simplifycfg, inline, ipscpp,
licm, loop-rotate, indvars, loop-unroll, loop-vectorize,
sccp, adce
```

First introduced at -02. Extra jump cleanup, redundancy elimination, SLP vectorisation, and a few CGSCC passes.

```
jump-threading, correlated-propagation, gvn, dse,
```

```
slp-vectorizer, aggressive-instcombine, constraint-elimination,
mldst-motion, speculative-execution, move-auto-init,
openmp-opt-cgsc
```

First introduced at -O3. Only three new names.

```
callsite-splitting, argpromotion, chr
```

3)

iii) transformations whose number or position of invocations changes between levels:

Most names that appear at -O1 keep the same count at -O2 and -O3. A few do not: they either run more often, or they already run more than once the first time they appear. The table is how many times each of those names showed up in our dump.

Pass	-O0	-O1	-O2	-O3
globaldce	1	2	2	2
licm	0	3	4	4
tailcallelim	0	1	2	2
vector-combine	0	1	2	2
jump-threading	0	0	2	2
correlated-propagation	0	0	2	2
instcombine	0	8	8	8
simplifycfg	0	8	8	8

`globaldce` already runs at -O0. From -O1 it runs twice: once after coroutine cleanup, and once near the end.

`licm` is 3 times at -O1 and 4 times from -O2, because O2 adds one more `licm` after `dse`.

`tailcallelim` is once at the end of O1. From O2 it also runs earlier, in the CGSCC block after the inliner.

`vector-combine` is once at O1. From O2 it runs after the mid-pipeline `sroa`, and again after `slp-vectorizer`.

`jump-threading` and `correlated-propagation` are not in O1 at all. When they start at O2 they already run twice: once before the loop nest and once after it.

`instcombine` and `simplifycfg` still run 8 times from O1 onwards. Other passes keep making new work for them, so they are run again and how often they run does not change. Where they sit in the list does because O2 and O3 put extra passes in between.

Those extra passes also slide most O1 names further down the list. The first five stay put, that is slots 1 to 5 are still `annotation2metadata`, `forceattrs`, `inferattrs`, `coro-early` and

	Pass	-O1 slots	-O2 slots	-O3 slots
	<code>mem2reg</code>	13	13	14
	<code>memcpyopt</code>	39	55	57
	<code>loop-vectorize</code>	61	76	79
	<code>loop-unroll</code>	68	84	87

`lower-expect`. After that the numbers move.

`instcombine` also moves, but it has eight places at each level, so that is a separate table:

Level	Slots
-O1	14, 22, 33, 42, 46, 64, 67, 71
-O2	14, 26, 40, 51, 61, 79, 83, 87
-O3	15, 28, 42, 53, 63, 82, 86, 90

`memcpyopt` is easy to see. At O1 it is right after the `sroa` that follows `loop-unroll-full` (slot 38, then 39). At O2 it comes after `adce`, because `gvn` and `jump-threading` run first.

The same name can also be given different options.

```
loop-unroll, loop-vectorize, simple-loop-unswitch
```

`loop-unroll` is printed as `loop-unroll<01>`, `loop-unroll<02>` or `loop-unroll<03>`. A higher level unrolls more.

`loop-vectorize` at O1 is marked `vectorize-forced-only`. At O2 and O3 that flag is off, so the loop is actually vectorised from O2.

`simple-loop-unswitch` at O1 and O2 only does the cheap (trivial) case. At O3 it is `<nontrivial;trivial>`, so it also tries the harder case.

4 Part D

`f1`, `f2` and `f3` are the three required loops in `vector.c`. `f4` is an extra interleave so that `shufflevector` also appears as a store shuffle, not only as the `alph` broadcast. `f5` returns the last value written in the loop, so the vectoriser has to pull one lane out with `extractelement`. Length is an argument. Array pointers that must not overlap are `restrict`. `main` uses buffers of 4096 elements and prints a checksum for five sizes.

Scalar IR first, then native vector IR (Homebrew LLVM 18.1.8). `-march=native` here means NEON, not AVX.

```
clang -O3 -fno-vectorize -fno-slp-vectorize -S -emit-llvm vector.c -o vector.scalar.ll
```

```
clang -O3 -march=native -Rpass=loop-vectorize -Rpass-missed=loop-vectorize \
-Rpass-analysis=loop-vectorize -S -emit-llvm vector.c -o vector.vector.ll \
2> vector.remarks.txt
```

With vectorisation off, each step is one scalar load / arithmetic / store. The remarks with it on are:

```
vector.c:9:  vectorized loop (vectorization width: 4, interleaved count: 4)
vector.c:17: vectorized loop (vectorization width: 4, interleaved count: 4)
vector.c:25: vectorized loop (vectorization width: 4, interleaved count: 4)
vector.c:34: vectorized loop (vectorization width: 4, interleaved count: 2)
vector.c:43: vectorized loop (vectorization width: 4, interleaved count: 4)
```

Those five lines are the `for` loops of `f1` through `f5` in the submitted `vector.c`. `f1` starts at line 9 because a one-line comment sits above the function. `f5` was added at the end, so it did not move the earlier loops.

Loop	What it does	Width	Interleave
<code>f1</code>	float $a[i] \times \text{alph} + b[i]$	4	4
<code>f2</code>	integer dot product	4	4
<code>f3</code>	pick $a[i]$ or $b[i]$ from <code>guess</code>	4	4
<code>f4</code>	write $a[i]$, $b[i]$ into even/odd slots	4	2
<code>f5</code>	$c[i] = 3a[i]$, return last $c[i]$	4	4

Width 4 is 128-bit NEON divided by a 32-bit `float` or `i32`. Interleave 4 means four of those vectors in the body, so 16 source iterations per trip of `f1`–`f3` and `f5`. The leftover tail is scalar, which is why $n = 7$ still works.

The vector types in `vector.vector.ll` are `<4 x float>`, `<4 x i32>`, `<4 x i1>` and `<8 x i32>`. `f5` reuses `<4 x i32>`; it does not add a new type.

Tracing f3

This is the load / compare / select / store walk. The C is `c[i] = guess[i] ? a_val : b_val` after both ints are loaded. One of the four interleaved steps looks like:

```
%18 = load <4 x i32>, ptr %14
%26 = load <4 x i32>, ptr %22
%34 = load <4 x i32>, ptr %30
%38 = icmp eq <4 x i32> %34, zeroinitializer
%42 = select <4 x i1> %38, <4 x i32> %26, <4 x i32> %18
store <4 x i32> %42, ptr %46
```

`%18` is four `a[i]`, `%26` is four `b[i]`, `%34` is four `guess[i]`. `icmp eq` against zero makes a `<4 x i1>` mask. `select` takes `b` when `guess` is 0, else `a`. Then four `c[i]` values are stored.

When f3 did not vectorise

The first body selected pointers, not values:

```
for (int i = 0; i < n; i++)
  c[i] = guess[i] ? a[i] : b[i];
```

`-Rpass-missed=loop-vectorize` printed nothing for this loop. The cost-model note only shows up with `-Rpass-analysis=loop-vectorize`. What clang did print:

```
vector.c: remark: the cost-model indicates that vectorization is not beneficial [-Rpass-analysis=loop-vectorize]
vector.c: remark: interleaved loop (interleaved count: 4) [-Rpass=loop-vectorize]
```

The IR then chose one pointer and loaded through it. One pass of that loop is:

```
%14 = icmp eq i32 %13, 0
%15 = select i1 %14, ptr %3, ptr %2
%16 = getelementptr inbounds i32, ptr %15, i64 %11
%17 = load i32, ptr %16, align 4
```

`%13` is `guess[i]`. If it is 0, `%15` is the `b` pointer; otherwise it is the `a` pointer. Then one integer is loaded through that pointer and stored to `c[i]`. There is no `<4 x i32> select`. An `if/else` that wrote `c[i]` got the same two remarks.

Only the loop body was changed. Loading both ints first, then using `?:` on the values, is the form that vectorises:

```
for (int i = 0; i < n; i++) {
  int a_val = a[i];
  int b_val = b[i];
  c[i] = guess[i] ? a_val : b_val;
}
```

That is the version in `vector.c` now, and the trace above.

f1: splat and FMA

`alph` is one scalar, so it is copied into every lane of a `<4 x float>` before the loop:

```
%12 = insertelement <4 x float> poison, float %4, i64 0
%13 = shufflevector <4 x float> %12, <4 x float> poison,
      <4 x i32> zeroinitializer
```

`insertelement` puts `alph` in lane 0. The shuffle mask is all zeros, so every lane becomes that same value.

Then each step is load / fused multiply-add / store:

```
%20 = load <4 x float>, ptr %16
%28 = load <4 x float>, ptr %24
%32 = call <4 x float> @llvm.fmuladd.v4f32(
      <4 x float> %20, <4 x float> %13, <4 x float> %28)
store <4 x float> %32, ptr %36
```

Four `a[i]`, four `b[i]`, the broadcast `alph`, four `c[i]` stored together. There are four copies of this because the interleave count is 4.

f2: integer reduction

`dot += a[i] * b[i]`. The loop keeps four `<4 x i32>` partial sums (the interleave). After the loop they are added and folded with:

```
%46 = call i32 @llvm.vector.reduce.add.v4i32(<4 x i32> %45)
```

That intrinsic is the horizontal add. There is no `extractelement`. Any leftover iterations use a scalar `phi`, same as the no-vectorise file.

Vector-specific opcodes

`insertelement`. Only the `alph` splat in f1.

`shufflevector`. The all-zero mask above is the broadcast. The other use is f4 (`c[2*i]=a[i]; c[2*i+1]=b[i];`):

```
%29 = shufflevector <4 x i32> %17, <4 x i32> %23,
      <8 x i32> <i32 0, i32 4, i32 1, i32 5,
               i32 2, i32 6, i32 3, i32 7>
store <8 x i32> %29, ptr %27
```

The mask is 0, 4, 1, 5, 2, 6, 3, 7. That order is `a0, b0, a1, b1, a2, b2, a3, b3`. The result is `<8 x i32>`, so eight ints go out in one store. Interleave 2 means two of these shuffles per trip.

`extractelement`. f2 never emits this. It folds the four partial sums with `llvm.vector.reduce.add.v4i32` instead of taking lanes out one by one. f4 did not change that; it only made another shuffle. So f5 stores `c[i] = 3 * a[i]` and returns the last `c[i]`. That last integer has to come out of the vector:

```
%23 = mul nsw <4 x i32> %19, <i32 3, i32 3, i32 3, i32 3>
store <4 x i32> %23, ptr %27
%31 = extractelement <4 x i32> %23, i64 3
```

Each trip of `f5` does sixteen `i` values (width 4, interleave 4). `%23` is the last four. Lane 3 is the last of those sixteen, so that is what the function returns when the whole `n` fitted in the vector loop. For a leftover tail, the scalar loop writes `last`, same as the C.

`select`. The vector `select <4 x i1>` in `f3`.

Correctness

```
clang -O3 -fno-vectorize -fno-slp-vectorize vector.c -o vector.scalar
clang -O3 -march=native vector.c -o vector.vector
```

Both programs printed the same `n` and checksum. 1024 and 2048 are at least 1024 elements. 7 and 1023 are not a multiple of 4, so the scalar tail ran as well.

<i>n</i>	checksum	scalar and vector match
7	1705808096	yes
16	2263840470	yes
1023	1403419796	yes
1024	2771994064	yes
2048	2356598585	yes

The checksum hashes the float bits (not a cast to `int`), plus the integer arrays, the dot product, the last value returned from `f5` and the `f5` stores. The two binaries printed the same five lines.